Win x64 Shellcode – Part 2: TEB, PEB and List of Loaded Modules

June 30, 2026 · 18 min

In the previous part, we explained why shellcode cannot use statically written addresses of Windows API functions. The solution lies in the structures that Windows maintains directly in the memory of each process. Today we will look at them closely.

Prerequisites

Before reading this part, it is advisable to read and understand the previous part. At the same time, it is highly advisable to have at least a basic understanding of what virtual memory and a pointer are.

TEB – Thread Environment Block

Each thread of a running process has a structure reserved in memory called **TEB (Thread Environment Block)**[1]. It is a key data structure that stores context information about a specific thread. Windows maintains it automatically and updates it at runtime, and it is directly accessible from user mode. Microsoft considers this structure internal, which corresponds with the fact that only part of this structure has a guaranteed layout. This public part is located at the very beginning of the `TEB` structure and bears the designation **NT_TIB**[2]. It serves primarily for the purposes of low-level development tools and applications that cannot work without up-to-date information about the thread (for example, exception handling compilers).

```
typedef struct _NT_TIB                                  // size 0x38
{
     PEXCEPTION_REGISTRATION_RECORD ExceptionList;      // offset 0x00
     PVOID StackBase;                                   // offset 0x08
     PVOID StackLimit;                                  // offset 0x10
```

```
        PVOID SubSystemTib;                             // offset 0x18
        union
        {
            PVOID FiberData;                            // offset 0x20
            ULONG Version;                              // offset 0x20
        };
        PVOID ArbitraryUserPointer;                     // offset 0x28
        PNT_TIB Self;                                   // offset 0x30
    } NT_TIB, *PNT_TIB;
```

To make the situation somewhat more complicated, there exists between offsets `0x38` and `0x68` a so-called "gray zone". The members of the structure in this range form pa of an internal structure that Microsoft kept secret for a long time, but due to backward compatibility and system tool developers, they had to keep their layout fixed. They are so crucial for application stability that it is not possible to change them arbitrarily. These are the following members:

- offset `0x38` - `EnvironmentPointer` (pointer to thread environment)
- offset `0x40` - `ClientId` (structure containing Process ID - PID - and Thread ID - TID)
- offset `0x50` - `ActiveRpcHandle` (handle of an actively running RPC call)
- offset `0x58` - `ThreadLocalStoragePointer` (pointer to thread local storage - key for TLS variables)
- offset `0x60` - `ProcessEnvironmentBlock` (pointer to `PEB` structure)
- offset `0x68` - `LastErrorValue` (last error code, which WinAPI `GetLastError()` internally uses)

On x64 architecture, the `TEB` of the current thread is always accessible via the `GS` segment register. From the aforementioned structure members, the offset `0x60` is particularly important for us, where the pointer to the `PEB` structure is located. The mentioned offset `0x60` represents the relative distance from the beginning of the `TEB` structure, not an absolute address in memory.

```
mov rax, gs:[0x60]   ; loads 8-byte address of PEB structure from TEB
```

In practice, you may still encounter another equivalent variant of obtaining `PEB`, which requires two instructions:

```
mov rax, gs:[0x30]    ; Loads from NT_TIB the pointer "Self" (linear address of TEB)
mov rax, [rax + 0x60] ; Adds offset 0x60 to this address and loads PEB
```

At first glance, this seems disadvantageous for shellcode, because the code will grow by several bytes. However, if we are not limited by the size of the exploited buffer (we have enough space), this approach is very suitable. The instruction `mov rax, gs:[0x30]` stores in the register the actual, absolute address of the `TEB` structure in RAM memory, which user code cannot otherwise read directly from the `GS` segment register. This opens doors for more advanced shellcodes that need to manipulate not only the `PEB` but also directly the thread context, such as managing `TLS` variables or low-level exception handling.

This procedure is key for all **Position Independent Code (PIC)** on the Windows x64 platform. It is a kind of universal entry point into the world of obtaining/discovering addresses at runtime (API resolution / runtime API resolution).

Using WinDbg, we can display the `TEB` structure with the `dt ntdll!_TEB` command.

```
0:000> !teb
TEB at 000000a94b722000
    ExceptionList:        0000000000000000
    StackBase:            000000a94b560000
    StackLimit:           000000a94b55b000
    SubSystemTib:         0000000000000000
    FiberData:            0000000000001e00
    ArbitraryUserPointer: 0000000000000000
    Self:                 000000a94b722000
    EnvironmentPointer:   0000000000000000
    ClientId:             0000000000017eb0 . 000000000001b250
    RpcHandle:            0000000000000000
    Tls Storage:          000001b1af317100
    PEB Address:          000000a94b721000
    LastErrorValue:       0
    LastStatusValue:      0
    Count Owned Locks:    0
    HardErrorMode:        0
```

(Notice that WinDbg shows the absolute address of `TEB` `000000a94b722000` . If the processor adds the relative offset `0x60` to this address, it reads the value

`000000a94b721000` at that location, which is the exact address of the `PEB` structure of the entire process, which we see on the `PEB` Address line)

PEB – Process Environment Block

PEB (Process Environment Block)[3] **represents the global state of the entire running process**. It is thus for the process what the `TEB` structure is for individual threads. While each thread has its own `TEB`, all threads within one process share one and the same `PEB`. It contains a wide range of information: from flags indicating the presence of a debugger, through the path to the executed `.exe` file, to process parameters. For the purposes of this text, however, the most important thing is the **list of all modules loaded into the process address space**.

The situation regarding the `PEB` structure from the perspective of guaranteeing its layout is very similar to that of `TEB`, but with one interesting difference. The vast majority of `PEB` remains officially undocumented. Microsoft, however, cannot afford to change its layout arbitrarily, because it would break a huge amount of existing commercial software that relies on it. In reality, internal changes occur almost constantly, but they concern members at very high offsets, where developers are no longer guaranteed anything.

An exception is formed precisely by the members `BeingDebugged` and `Ldr`, which matter most to us in this text. These are, unlike the rest of the structure, officially documented, although this happened somewhat involuntarily. Microsoft apparently in 2002 as part of a settlement in American antitrust proceedings published a header file `winternl.h` [4], containing a trimmed version of the `PEB` structure with the `BeingDebugged` member at its actual offset, while filling the rest at that time with anonymous padding to preserve correct positions in memory. The members `Ldr` and `ProcessParameters` were added to this header file later, specifically in the SDK for Windows 7. This "official" version of `PEB` is still fully available in Microsoft Learn documentation.

For this reason, today a large part of low-level developers prefer to access values in this structure directly instead of calling standard Windows API functions. An example is precisely the `BeingDebugged` member at offset `0x02`, from which the documented function `IsDebuggerPresent()` from `kernel32.dll` reads data. For the rest of the hidden fields, the consistency of layout can be described as an "unwanted" guarantee of backward compatibility on Microsoft's part. For these two members, however, the "guarantee" is completely official.

Because the `PEB` structure is in reality extremely extensive[5], we will present only the members relevant for this document:

```
typedef struct  _PEB
{
        ...
        UCHAR BeingDebugged;            // offset 0x02
        ...
        PVOID ImageBaseAddress;         // offset 0x10
        PPEB_LDR_DATA Ldr;                      // offset 0x18
        ....
} PEB, *PPEB;
```

Using WinDbg, we can display the `PEB` structure with the `dt ntdll!_PEB` command.

At the key offset `0x18` is located the sought pointer to the `PEB_LDR_DATA` structure.

PEB_LDR_DATA and Module Lists

The `PEB_LDR_DATA` structure[6] is one of the most important members of the `PEB` structure. Through this node, the program loader in ntdll.dll manages and records all dynamic libraries that the given process has loaded into its address space. Officially, this structure is documented only partially, but its internal layout is stable on x64 architecture due to compatibility.

```
typedef struct _PEB_LDR_DATA
{
     ULONG Length;                                  // offset 0x00
```

```
    UCHAR Initialized;                          // offset 0x04
    PVOID SsHandle;                             // offset 0x08
    LIST_ENTRY InLoadOrderModuleList;           // offset 0x10
    LIST_ENTRY InMemoryOrderModuleList;         // offset 0x20
    LIST_ENTRY InInitializationOrderModuleList; // offset 0x30
    PVOID EntryInProgress;                      // offset 0x40
} PEB_LDR_DATA, *PPEB_LDR_DATA;
```

Using WinDbg, we can display the `PEB_LDR_DATA` structure with the `dt ntdll!_PEB_LDR_DATA` command.

From the aforementioned members, we are interested in three doubly-linked lists containing loaded modules. Each of these lists orders the same modules in a different order:

- `InLoadOrderModuleList` orders libraries according to how the system loader loaded them. The executable file of the program itself is usually on the first positions, followed by `ntdll.dll` and other dependencies.
- `InMemoryOrderModuleList` orders libraries according to their actual placement in the RAM address space.
- `InInitializationOrderModuleList` orders libraries according to the order in which their initialization functions were executed (i.e., the call to the `DllMain` function). Because the main executable file of the application (`.exe`) is not initialized as a DLL, you will not find it in this list at all. At the first positions here, usually purely system libraries such as `ntdll.dll` and `kernel32.dll` reside due to dependencies.

For our purposes, it is most suitable to use `InMemoryOrderModuleList` , because it has historically been the most consistent across versions of Windows and at the same time it allows us to very easily identify the first two libraries in memory. These are `ntdll.dll` (the second module in order, immediately after the running application itself) and `kernel32.dll` (the third module in order). Each of these lists is formed as a circular doubly-linked list, where each node represents a `LIST_ENTRY` [7] structure:

```
typedef struct _LIST_ENTRY
{
   struct _LIST_ENTRY *Flink;   // offset 0x00
   struct _LIST_ENTRY *Blink;   // offset 0x08
```

```
} LIST_ENTRY, *PLIST_ENTRY;
```

Using WinDbg, we can display the `LIST_ENTRY` structure with the `dt ntdll!_LIST_ENTRY` command.

LDR_DATA_TABLE_ENTRY

Each loaded module is represented by a `LDR_DATA_TABLE_ENTRY` [8] structure. For us, these fields are relevant:

```
LDR_DATA_TABLE_ENTRY
├── InMemoryOrderLinks      (offset 0x00)  –  LIST_ENTRY (Flink, Blink)
├── ...
├── DllBase                 (offset 0x20)  –  base address of the module in memory
├── ...
├── BaseDllName             (offset 0x58)  –  UNICODE_STRING (file name)
└── ...
```

Using WinDbg, we can display the `LDR_DATA_TABLE_ENTRY` structure with the `dt ntdll!_LDR_DATA_TABLE_ENTRY` command.

`InMemoryOrderLinks` is a `LIST_ENTRY` structure containing two pointers:

- `Flink` – pointer to the next node in the list
- `Blink` – pointer to the previous node in the list

By traversing the `Flink` pointers, we progressively go through all loaded modules.

```
; ===================================================================
; Finding kernel32.dll using fixed position (index) in memory
; ===================================================================

mov rax, gs:[0x60]       ; RAX = Address of PEB from GS register
mov rax, [rax + 0x18]    ; RAX = Address of Ldr (PEB_LDR_DATA)
mov rsi, [rax + 0x20]    ; RSI = Address of the head of InMemoryOrderModuleList

; --- Blind jumping through nodes (assumption of fixed order) ---
```

```
    lodsq                   ; 1. The executable file of the process itself
    xchg rax, rsi

    lodsq                   ; 2. The ntdll.dll library
    xchg rax, rsi

    lodsq                   ; 3. Expected position of kernel32.dll

    ; --- Calculation of the base address ---

    mov rbx, [rax + 0x20]    ; RBX = Address of DllBase of the kernel32.dll library
```

Blind reliance on a fixed order (indices) of modules, that is, the assumption that `kernel32.dll` will always be exactly the third link in the chain, is ideal for understandin the basic principle and shortening the sample code. In Windows history, this order did change (for example, in the Windows 9x/Millennium era, where `kernel32.dll` was sometimes the fifth or sixth), but on modern NT systems, these key libraries are preser in memory practically immediately. The `ntdll.dll` library is mapped by the OS kernel itself as the very first, and `kernel32.dll` sits at the very top of the dependency import tree.

Nevertheless, even on modern Windows 10 and 11, there is a risk that a blind shellcode without name verification will crash. The introduction of parallel loader (`ntdll!LdrpEnableParallelLoading`) causes the order of completion of library mapping to be slightly shaken. However, a much greater risk is posed by security tools (EDR/AV) or game protection, which inject themselves into the new process right at startup. These foreign libraries, possibly companion modules such as `KERNELBASE.dll` or `kernel.appcore.dll` , can overtake each other in the list and occupy precisely that expected third position.

If we want our shellcode to be truly reliable, it is necessary to add active name verification from the `BaseDllName` field to the search loop. We can start with direct byte comparison of the specific text string (for example, by verifying the entire word `"kernel32"` in memory using 64-bit registers).

```
    ; ====================================================================
    ; Robust finding of kernel32.dll via InMemoryOrderModuleList (x64)
    ; ====================================================================
```

```
mov rax, gs:[0x60]          ; Address of PEB from GS register
mov rax, [rax + 0x18]       ; RAX = Address of Ldr (PEB_LDR_DATA)
mov rdi, [rax + 0x20]       ; RDI = Head pointer to InMemoryOrderModuleList (start/end)
mov rsi, [rdi]              ; RSI = First node of the list (Flink)

find_dll_loop:
    cmp rsi, rdi            ; Have we already checked the entire circular list?
    je dll_not_found        ; If yes, we're back at the beginning -> end

    ; --- Verification of string length in BaseDllName ---
    ; The BaseDllName field is actually a UNICODE_STRING structure and starts at offset
    ; The first 16 bits (WORD) is the Length field.
    ; For the string "kernel32.dll" (12 characters * 2 bytes in UTF-16), the length mus

    mov cx, [rsi + 0x48]    ; CX = BaseDllName.Length
    cmp cx, 0x18            ; Does the name have a length of 24 bytes ("kernel32.dll")?
    jne next_dll            ; If not, continue to the next DLL

    ; --- Verification of the actual name content ---
    ; The pointer to the text string (Buffer) is located at offset +0x08 from the begin
    mov rdx, [rsi + 0x50]   ; RDX = Pointer to text Buffer

    ; Preparation of normalization mask for conversion to lowercase letters
    mov r9, 0x0020002000200020

    ; 1. Block verification: "kern"
    mov rax, [rdx]
    or rax, r9              ; Convert all characters in RAX to lowercase letters
    mov r8, 0x006e00720065006b ; Little Endian for "kern"
    cmp rax, r8
    jne next_dll            ; If mismatch, continue to the next DLL

    ; 2. Block verification: "el32"
    mov rax, [rdx + 8]
    or rax, r9              ; Convert to lowercase letters
    mov r8, 0x00320033006c0065 ; Little Endian for "el32"
    cmp rax, r8
    jne next_dll            ; If mismatch, continue to the next DLL

    ; 3. Block verification: ".dll"
    mov rax, [rdx + 16]
    or rax, r9              ; Convert to lowercase letters
    mov r8, 0x006c006c0064002e ; Little Endian for ".dll" adjusted for dot position aft
    cmp rax, r8
    je dll_found            ; If both the length and all 24 normalized bytes match -> f
```

```
next_dll:
    mov rsi, [rsi]          ; Move to next node
    jmp find_dll_loop

dll_found:
    ; --- We found the correct node ---
    ; The DllBase field is located at offset +0x20 from InMemoryOrderLinks.
    mov rbx, [rsi + 0x20]   ; RBX = Base address of kernel32.dll
    jmp end

dll_not_found:
    xor rbx, rbx            ; Library was not found, we zero RBX

end:
    ; In register RBX we now have safely stored the address of the beginning of kernel3
```

When writing a search loop, it is not enough to only check the contents of memory. What if a longer string lay in memory that coincidentally starts the same – for example `kernel32.dll.tmp` ? Our 64-bit registers would read the first 24 bytes, report a match, and ignore the fact that the string in memory continues further. This would lead to a fatal crash.

To prevent this, we must use the properties of the UNICODE_STRING structure[9], of which the BaseDllName field is composed. This structure has at its very beginning (offset `0x58`) a 16-bit Length field that stores the exact length of the name in bytes. Because we access the list via InMemoryOrderModuleList, this field is located at relativ offset `0x48` . And because we know that "kernel32.dll" has exactly 12 characters, in UTF 16LE encoding it must have a length of exactly 24 bytes (0x18). The first step of our loop is therefore a lightning verification of this length. If it doesn't match, the code immediately skips to the next module without wasting time comparing text. This gives us absolute certainty of string termination.

Windows, furthermore, ignores case, so in memory we can encounter `kernel32.dll` , `KERNEL32.DLL` , or `Kernel32.Dll` . If we compare entire 64-bit registers at once, it seems like an unsolvable problem. We certainly won't analyze the string character by character, will we?

Fortunately, in the ASCII/Unicode table, uppercase and lowercase letters have an identical binary representation except for a single bit (with value `0x20`). If we apply a logical `OR` instruction with a mask `0x0020002000200020` to the loaded register, for all uppercase letters we enforce this bit to value `1`. An uppercase K becomes lowercase k, while already lowercase letters remain unchanged.

What happens to the numbers and dot in the `.dll` extension? The logical `OR` operation will of course affect the binary value of some of these characters and distort the text for the human eye. But this doesn't matter at all, because we're not searching for human-readable text in assembly. It's enough for us that this operation creates in the register an absolutely constant and predictable footprint regardless of whether the original name was entered in uppercase or lowercase letters. We've adjusted our comparison constant in register `R8` so that it already accounts for this resulting state after the `OR` operation in advance. We thus compare visually distorted, but one hundred percent unified data, and that within a single processor clock cycle without any code branching.

An even more advanced and in practice most widespread variant is then the use of hashing. Instead of storing entire text names of libraries (which unnecessarily take up space and attract the attention of antivirus software), the shellcode calculates a short numerical hash of the name of each library at runtime and compares it with a pre-prepared value. Only in this way can we ensure one hundred percent stability of the shellcode.

```
; ====================================================================
; Finding kernel32.dll using ROR13 hashing algorithm
; ====================================================================

mov rax, gs:[0x60]          ; RAX = Address of PEB from GS register
mov rax, [rax + 0x18]       ; RAX = Address of Ldr (PEB_LDR_DATA)
mov rdi, [rax + 0x10]       ; RDI = Head pointer to InLoadOrderModuleList (start/end)
mov rsi, [rdi]              ; RSI = First node of the list (Flink)

find_dll_loop:
    cmp rsi, rdi            ; Have we already checked the entire circular list?
    je dll_not_found        ; If yes, we're back at the beginning -> end

    ; --- Verification of pointer validity to name ---
```

```
    ; At offset +0x60 from the beginning of the node lies a pointer to the text string
    mov rdx, [rsi + 0x60]    ; RDX = Pointer to Unicode text string (Buffer)
    test rdx, rdx            ; Is the Buffer pointer null (NULL)?
    jz next_dll              ; If yes, we skip the module (crash protection)

    ; --- Preparation for hash calculation from BaseDllName ---
    ; The BaseDllName field starts at offset 0x58.
    ; The first 16 bits (WORD) is the Length field, which indicates the length in bytes
    movzx ecx, word [rsi + 0x58] ; ECX = BaseDllName.Length (number of bytes)
    xor r9, r9               ; R9 = We will accumulate the resulting hash here (starting

hash_loop:
    ; --- The ROR13 algorithm itself ---
    ror r9d, 13              ; Rotate the current hash right by 13 bits

    ; Load 1 byte (Unicode character has 2 bytes, reading the first byte performs ASCII
    movzx eax, byte [rdx]

    ; --- Convert to lowercase letters (Case Insensitivity) ---
    ; Because Windows can return the name as "KERNEL32.DLL" as well as "kernel32.dll",
    ; we convert uppercase letters (A-Z) to lowercase (a-z) by adding 0x20.
    cmp al, 'A'
    jl skip_lowercase
    cmp al, 'Z'
    jg skip_lowercase
    add al, 0x20             ; Convert to lowercase letter

skip_lowercase:
    add r9d, eax             ; Add current normalized character to the hash

    add rdx, 2               ; Move to the next character in the Unicode string (+2 byte
    sub ecx, 2               ; Subtract 2 bytes from the total length of the name
    jnz hash_loop            ; Repeat until we process all bytes of the length

hash_finished:
    ; --- Hash value comparison ---
    ; The value 0x8FECD63F is a pre-calculated ROR13 hash for the string "kernel32.dll"
    cmp r9d, 0x8FECD63F
    je dll_found             ; If the hashes match, we found kernel32.dll!

next_dll:
    mov rsi, [rsi]           ; Move to next node (RSI = current_node->Flink)
    jmp find_dll_loop        ; Repeat main loop

dll_found:
    ; --- We found the correct node ---
```

```
    ; In the InLoadOrderModuleList list, the base address (DllBase) is located at offse
    mov rbx, [rsi + 0x30]   ; RBX = Base address of kernel32.dll
    jmp end

dll_not_found:
    xor rbx, rbx            ; Library was not found, we zero RBX

end:
    ; In register RBX we now have safely stored the address of the beginning of kernel3
```

Note on UNICODE_STRING

Module names are stored in the `UNICODE_STRING` format, that is, as a UTF-16LE string. The structure has the following definition:

```
typedef struct _UNICODE_STRING {
    USHORT Length;        // Current length of the string in bytes (not in characters!)
    USHORT MaximumLength; // Maximum capacity of allocated memory in bytes
    PWSTR  Buffer;        // Pointer to the actual field of wide characters (WCHAR*)
} UNICODE_STRING, *PUNICODE_STRING;
```

Using WinDbg, we can display the `UNICODE_STRING` structure with the `dt ntdll!_UNICODE_STRING` command.

Summary

- `TEB` is accessible via `GS:[0x60]` and contains a pointer to `PEB`
- `PEB` at offset `0x18` contains a pointer to `PEB_LDR_DATA`
- `PEB_LDR_DATA` at offset `0x20` contains the head of the `InMemoryOrderModuleList` list
- Each node of the list is actually a `LDR_DATA_TABLE_ENTRY` structure with the base address and name of the module
- By traversing the `Flink` pointers we find any loaded module in memory

What’s coming in the next part

We know how to find the base address of a module. But the base address alone is not enough – we need the address of a specific function. In the next part, we will look at the PE structure and Export Directory, where Windows stores a table of exported functions for each module.

Reference

- [1] TEB (Thread Environment Block) - geoffchappell.com
- [2] struct NT_TIB - Nirsoft
- [3] PEB (Process Environment Block) - Microsoft Learn
- [4] PEB - geoffchappell.com
- [5] strukct PEB - geoffchappell.com
- [6] struct PEB_LDR_DATA - geoffchappell.com
- [7] struct LIST_ENTRY - Microsoft Learn
- [8] struct LDR_DATA_TABLE_ENTRY - geoffchappell.com
- [9] struct UNICODE_STRING - geoffchappell.com

Shellcode | Windows | Assembly | Windows-Internals | X64 | Exploit-Dev

« PREV
Win x64 Shellcode – Part 1: Why We Can't Use Static Addresses

NEXT »
Interactive PE Header Map: A Visual Guide to Binary Structure

© 2026 proteqtum · Powered by Hugo & PaperMod